

AI-Powered Hospital Flow Management System

Database Architecture & SQLAlchemy Implementation Reference

Backend	ORM	Database	AI/ML
Flask	Flask-SQLAlchemy	MySQL / PostgreSQL	Python + Scikit-learn

Purpose

This document is the database engineer's implementation reference. The system uses Flask + SQLAlchemy with a relational database and does not use Firestore.

High-Level Structure

Hospital -> Doctors, Appointments, Queue Events, ML Predictions

Patient -> Appointments, Medical Records

Appointment -> Queue Entry

Doctor -> Availability, Appointments, Queue Events, Medical Records, ML Predictions

Relationship Map

Parent	Rel.	Child	Purpose
Hospital	1:N	Doctor	Doctors working at hospital
Hospital	1:N	Appointment	Hospital appointments
Hospital	1:N	QueueEvent	Hospital flow events
Hospital	1:N	MLPrediction	Flow predictions
Doctor	1:N	Appointment	Doctor appointments
Doctor	1:N	DoctorAvailability	Availability / breaks
Doctor	1:N	QueueEvent	Doctor flow events
Doctor	1:N	MedicalRecord	Doctor-authored records
Doctor	1:N	MLPrediction	Doctor predictions
Patient	1:N	Appointment	Appointment history
Patient	1:N	MedicalRecord	Medical history
Appointment	1:1*	QueueEntry	Current queue record

* MVP: one current queue entry per appointment. Historical queue states can later use 1:N.

Database Tables - Core Entities

1. hospitals

Column	Type	Key / Rule	Description
hospital_id	Integer	PK	Unique hospital identifier
name	String	NOT NULL	Hospital name
location	String		Hospital location
contact_number	String		Contact number
created_at	DateTime	DEFAULT	Creation timestamp

2. patients

Column	Type	Key / Rule	Description
patient_id	Integer	PK	Unique patient identifier
name	String	NOT NULL	Patient name
email	String	UNIQUE	Email
phone	String		Phone
date_of_birth	Date		Date of birth
gender	String		Gender
address	String		Address
created_at	DateTime	DEFAULT	Creation timestamp

3. doctors

Column	Type	Key / Rule	Description
doctor_id	Integer	PK	Unique doctor identifier
hospital_id	Integer	FK -> hospitals	Hospital
name	String	NOT NULL	Doctor name
specialization	String		Specialization
department	String		Department
status	String		AVAILABLE / BUSY / ON_BREAK / UNAVAILABLE
created_at	DateTime	DEFAULT	Creation timestamp

4. appointments

Column	Type	Key / Rule	Description
appointment_id	Integer	PK	Unique appointment
patient_id	Integer	FK -> patients	Patient
doctor_id	Integer	FK -> doctors	Doctor
hospital_id	Integer	FK -> hospitals	Hospital
appointment_date	Date	NOT NULL	Appointment date
appointment_time	Time	NOT NULL	Appointment time

Column	Type	Key / Rule	Description
reason	Text		Reason
status	String		PENDING / APPROVED / REJECTED / CANCELLED / COMPLETED / NO_SHOW
created_at	DateTime	DEFAULT	Creation time
updated_at	DateTime	DEFAULT	Last update

Database Tables - Queue & Flow

5. queue_entries

Column	Type	Key / Rule	Description
queue_entry_id	Integer	PK	Unique queue record
appointment_id	Integer	FK -> appointments	Appointment
patient_id	Integer	FK -> patients	Patient
doctor_id	Integer	FK -> doctors	Doctor
hospital_id	Integer	FK -> hospitals	Hospital
queue_position	Integer		Current position
status	String		WAITING / READY / CALLED / IN_CONSULTATION / COMPLETED / CANCELLED
estimated_wait_time	Float		Estimated wait in minutes
priority_type	String		Operational priority classification
joined_at	DateTime		Queue join
called_at	DateTime		Called time
consultation_started_at	DateTime		Consultation start
consultation_completed_at	DateTime		Completion time

6. doctor_availability

Column	Type	Key / Rule	Description
availability_id	Integer	PK	Unique availability record
doctor_id	Integer	FK -> doctors	Doctor
hospital_id	Integer	FK -> hospitals	Hospital
status	String		AVAILABLE / BREAK / UNAVAILABLE
start_time	DateTime		Status start
end_time	DateTime		Status end
reason	String		Optional reason
created_at	DateTime	DEFAULT	Creation timestamp

7. queue_events

Column	Type	Key / Rule	Description
event_id	Integer	PK	Unique event
hospital_id	Integer	FK -> hospitals	Hospital
doctor_id	Integer	FK -> doctors	Doctor if applicable
appointment_id	Integer	FK -> appointments	Appointment if applicable
queue_entry_id	Integer	FK -> queue_entries	Queue record if applicable
event_type	String	NOT NULL	PATIENT_JOINED / COMPLETED / EMERGENCY_ADDED / BREAK / UNAVAILABLE
event_time	DateTime	NOT NULL	Event time

Column	Type	Key / Rule	Description
duration	Float		Duration in minutes
description	Text		Additional context
created_at	DateTime	DEFAULT	Creation timestamp

Database Tables - Medical & ML

8. medical_records

Column	Type	Key / Rule	Description
record_id	Integer	PK	Unique medical record
patient_id	Integer	FK -> patients	Patient
doctor_id	Integer	FK -> doctors	Doctor
appointment_id	Integer	FK -> appointments	Consultation
diagnosis	Text		Diagnosis
symptoms	Text		Symptoms
medications	Text		Medications / prescription
allergies	Text		Known allergies
notes	Text		Doctor notes
created_at	DateTime	DEFAULT	Creation timestamp

9. ml_predictions

Column	Type	Key / Rule	Description
prediction_id	Integer	PK	Unique prediction
hospital_id	Integer	FK -> hospitals	Hospital
doctor_id	Integer	FK -> doctors	Doctor if applicable
prediction_type	String		WAIT_TIME / LOAD / QUEUE_LENGTH
queue_size	Integer		Queue size input
doctors_available	Integer		Available doctor input
emergency_cases	Integer		Emergency count input
appointments_count	Integer		Appointment count input
avg_consultation_time	Float		Average consultation time
predicted_wait_time	Float		Predicted waiting time
predicted_load	String		LOW / MEDIUM / HIGH
model_version	String		Model version
prediction_time	DateTime	NOT NULL	Prediction timestamp

Operational Data Flows

- A. Appointment: Patient -> Appointment -> Receptionist approval -> QueueEntry -> QueueEvent -> ML prediction -> dashboard.
- B. Consultation completion: Consultation completed -> Appointment/QueueEntry updated -> QueueEvent -> next eligible patient -> ML re-prediction.
- C. Emergency/delay: Authorized staff records event -> QueueEvent -> operational rules update flow -> ML predicts downstream impact.
- D. Doctor break: DoctorAvailability updated -> affected queue recalculated -> ML prediction refreshed.

ML Integration Contract

ML reads current operational data through the Flask/SQLAlchemy service layer. ML predicts flow impact; backend rules and authorized hospital staff remain responsible for operational and clinical actions.

Inputs: patients waiting, doctors available, emergency cases, appointments count, average consultation time, time features and validated flow features.

Outputs: predicted waiting time, predicted load/queue length, model version and prediction timestamp.

Implementation Rules

- Use a relational database with SQLAlchemy foreign keys; do not use Firestore.
- Maintain one canonical schema shared by all team members.
- Index frequently queried foreign keys and operational fields such as appointment status, queue status, doctor_id, hospital_id and event_time.
- Keep event timestamps/history so historical flow data can support ML training.
- Do not hard-code the Top-5 preparation window in the database; keep it configurable as an application/business rule.
- Emergency clinical priority remains under authorized hospital/clinical rules; ML predicts flow impact.
- Store model_version with predictions for traceability.
- Use Flask-Migrate/Alembic for team schema migrations once stable; db.create_all() is mainly for early prototypes.

Recommended Build Order

Phase	Tables	Goal
1	hospitals, patients, doctors	Core entities
2	appointments, queue_entries	Appointments and queue
3	doctor_availability, queue_events	Dynamic hospital flow
4	medical_records	Patient history
5	ml_predictions	ML integration and prediction storage